

Interview Q&A Master Reference

Senior Full Stack Developer
Ankit

.NET Core · Angular · Azure · SQL Server · Docker

10+
Companies

140+
Q&A Pairs

9 yrs
Experience

2026
Edition

Wipro	Round 1 · Round 2 · L1 (18 Jun 2026)
Successive Noida	Round 1 (14 May 2026)
Decos Global	Round 1 (2 Jun 2026)
HCL	Round 1 (8 Jun 2026)
HCL – Optum	Round 1 (Jun 2026)
Infosys	Round 1 (Jun 2026)
Virtusa	Round 1 (11 Jun 2026)
CitiusTech	Round 1 · 26 Jun 2026
NeoSoft ★	Round 1 · 14 Jul 2026 — LATEST
Persistent Systems ★	Round 1 · 14 Jul 2026 — LATEST

CitiusTech

Round 1 — Technical Interview · 26 June 2026



These are the actual questions asked on 26 June 2026 — the most recent interview. Study these first. All Q&A cards in this chapter are highlighted in amber.

Introduction & Background

Q01

General & Intro

Could you please give a brief introduction — experience, tech skills, recent project?

ANSWER

Decision: lead with years, specialisation, last client, one measurable win. Stay under 90 seconds.

I'm Ankit, a Senior Full Stack Developer with 9 years of experience specialising in C# .NET Core, Angular, React, and Azure. My most recent engagement was with Grant Thornton (healthcare domain), where I built an Entity Management System — tracking business entities, shareholding percentages, and partnerships.

Tech in action: .NET Core 8 Web API with Clean Architecture (Controller → Service → Repository → EF Core → SQL Server), Angular 17 standalone components, and Azure (App Service, Functions, Service Bus, Blob Storage, DevOps CI/CD, Application Insights).

Key impact: our dashboard was timing out at ~8 seconds due to N+1 queries. I rewrote the logic as a single CTE-based stored procedure → response time dropped to 420ms (~95% improvement), verified via SET STATISTICS TIME ON and Application Insights.

INTERVIEW TIP: Mention GitHub Copilot usage to show AI-augmented productivity. Keep it conversational, not rehearsed.

Q02

Azure DevOps / CI/CD

Those CI/CD pipelines in Azure DevOps — did you create them yourself?

ANSWER

Yes — I personally created and maintained the Azure DevOps YAML pipelines end-to-end.

CI Pipeline: triggered on PR/merge to main. Steps: dotnet restore → dotnet build → dotnet test (unit + integration) → publish artifact → upload to pipeline drop.

CD Pipeline: deploy to staging slot → smoke test (/health endpoint) → manual approval gate → swap staging to production slot (zero downtime).

Docker: docker build → tag with Build.BuildId → push to Azure Container Registry.

I also configured: branch policies (PRs require passing CI + 1 approver), environment gates (production = 2 approvals), variable groups for environment config — secrets pulled from Azure Key Vault via managed identity,

never hardcoded in YAML.

INTERVIEW TIP: Say "Yes, I owned them end-to-end." Mention YAML pipelines (not classic GUI editor) — signals modern DevOps awareness.

Q03

.NET Core

Which version of .NET Core are you currently working in?

ANSWER

.NET 8 (LTS — Long-Term Support, released November 2023).

Key .NET 8 features I actively use:

Minimal APIs with route groups — cleaner, faster-starting lightweight endpoints.

Primary constructors — cleaner DI injection syntax.

Keyed services in DI container — register multiple implementations of same interface with a named key.

IExceptionHandler interface — cleaner than custom middleware for exception handling.

Native AOT support — reduces cold-start time (critical for Azure Functions on Consumption plan).

Performance: LINQ improvements, better Span<T> optimisations, improved regex compilation.

Previous: .NET Core 3.1, .NET 6 LTS. Migrated a project from .NET 6 → .NET 8 and observed ~15% throughput improvement with zero code changes.

INTERVIEW TIP: Naming one or two .NET 8-specific features proves you are current, not just citing a version number.

Q04

Angular

And which version of Angular are you working with?

ANSWER

Angular 17 (with familiarity of Angular 18 preview features).

Key Angular 17 features I use in production:

Standalone Components — no NgModule. Components declare their own imports, cleaner and more tree-shakeable.

@if / @for / @switch — new built-in control flow blocks replacing *ngIf and *ngFor. Better performance and readability.

Signals — fine-grained reactivity eliminating Zone.js overhead for targeted reactive patterns.

Deferrable Views (@defer) — lazy-load heavy UI blocks only when they enter the viewport.

esbuild by default — 2–5x faster builds than Webpack in our experience.

Previous: Angular 12, 14, 15, 16. Strong RxJS background with interceptors, guards, reactive forms.

INTERVIEW TIP: Mention Signals — it is the most discussed Angular 17 feature and shows you track framework evolution.

Q05

Database

What database are you using on the database side?

ANSWER

Primary: SQL Server (Azure SQL Database in production — managed, scalable, built-in HA and geo-replication).

How I use it:

EF Core 8 Code-First with migrations for OLTP CRUD operations.

Stored procedures for complex aggregations (shareholding recalculation across 4 tables).

Indexed views and covering indexes for dashboard read-heavy queries.

Query Store + Application Insights for ongoing performance monitoring.

Also experienced with:

MongoDB / Cosmos DB API — semi-structured document storage where schema flexibility was needed.

Redis — distributed caching of reference data with sliding expiration.

SQL features I use regularly: CTEs, window functions (ROW_NUMBER, DENSE_RANK), OFFSET/FETCH pagination, execution plans, SQL Profiler, Extended Events.

Q06

Cloud / Azure

Any exposure to cloud platforms?

ANSWER

Yes — Azure is my primary cloud platform, with hands-on production experience across a wide set of services.

Azure App Service — hosted .NET Core Web APIs with deployment slots (staging → production swap for zero downtime).

Azure Functions — serverless background processing with HTTP Trigger and Service Bus Trigger.

Azure Service Bus — async messaging between microservices. Topics + Subscriptions for fan-out event patterns.

Azure Blob Storage — document storage with SAS tokens for secure client-direct uploads.

Azure SQL Database — managed SQL Server with geo-redundant backups and Query Performance Insight.

Azure DevOps — full CI/CD YAML pipelines (build → test → deploy → gate → swap).

Application Insights — distributed tracing, custom events, P95 latency alerts.

Azure Key Vault — secrets management via managed identity. No secrets in code or config files.

Azure API Management — gateway for microservices routing, auth, and rate limiting.

Azure Container Registry (ACR) — Docker image repository for microservice deployments.

Azure Container Apps — serverless container hosting without managing Kubernetes.

INTERVIEW TIP: Structure by category — compute, storage, messaging, DevOps, monitoring. Shows systematic thinking.

Q07

Docker / Kubernetes

Any exposure to Docker or Kubernetes?

ANSWER

Yes — Docker is part of my daily workflow. Kubernetes (AKS) I have used at the configuration/deployment level.

Docker experience:

Multi-stage Dockerfiles for .NET Core microservices — SDK image for build, lean ASP.NET runtime for deployment (~200MB final vs ~700MB SDK).

.dockerignore to exclude bin/obj/secrets from build context.

docker-compose.yml for local dev — spin up API + SQL Server + Service Bus emulator with one command.

Security: containers run as non-root user (USER app in Dockerfile).

Kubernetes / AKS:

Deployed containerised microservices to Azure Kubernetes Service.

Wrote Kubernetes YAML manifests: Deployment, Service, ConfigMap, Secret.

Used Azure Container Apps on recent projects — serverless Kubernetes without managing the control plane.

Familiar with: rolling deployments, health probes, horizontal pod autoscaling, namespaces.

GitHub Copilot: I use Copilot to generate Dockerfile starters and K8s manifests, then review and customise — saves ~30–40% of boilerplate time.

Q08

Docker / .NET Core

As a 9-year experienced developer: steps to create a new .NET Core app and deploy/run it from Docker?

ANSWER

STEP 1 — Create the app:

```
dotnet new webapi -n MyApp --use-controllers
```

```
cd MyApp && dotnet run
```

STEP 2 — Write the Dockerfile (multi-stage, always):

```
# Stage 1: Build
```

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
```

```
WORKDIR /src
```

```
COPY ["MyApp.csproj", "."]
```

```
RUN dotnet restore
```

```
COPY . .
```

```
RUN dotnet publish -c Release -o /app/publish
```

```
# Stage 2: Runtime (lean ~200MB image)
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
EXPOSE 8080
COPY --from=build /app/publish .
USER app
ENTRYPOINT ["dotnet", "MyApp.dll"]
```

STEP 3 — Add .dockerignore (exclude bin/, obj/, secrets)

STEP 4 — Build: `docker build -t myapp:v1 .`

STEP 5 — Run: `docker run -d -p 8080:8080 --name myapp myapp:v1`

STEP 6 — Verify: `curl http://localhost:8080/weatherforecast`

STEP 7 — Push to ACR: `docker push myacr.azurecr.io/myapp:$(Build.BuildId)`

Key decisions: multi-stage (lean final image), USER app (non-root security), EXPOSE 8080 (.NET 8 default non-root port — not 80), .dockerignore reduces context and avoids shipping secrets.

Q09

Azure Functions

What is the Consumption Plan in Azure Function App? What is its purpose?

ANSWER

The Consumption Plan is the default serverless hosting plan for Azure Functions.

How it works: you pay only for executions — billed per invocation plus GB-second of memory used. Azure automatically allocates and deallocates compute based on trigger demand. Scales to zero when idle (zero cost when not running). Auto-scales up to 200 concurrent instances.

Purpose — when to use it:

Event-driven workloads with unpredictable or sporadic traffic.

Background jobs (timer-based, queue-triggered) that do not run continuously.

Cost optimisation — you pay nothing when there is no activity.

Free grant: 1 million executions + 400,000 GB-seconds per month.

Key characteristics:

Execution timeout: 5 minutes default (max 10 minutes on Consumption).

Cold start: first invocation after idle takes 100ms–2s because Azure provisions a new instance.

Consumption vs other plans:

Premium Plan — no cold start (pre-warmed instances), VNET integration, longer timeout. Use for latency-sensitive workloads.

Dedicated Plan — always-on, fixed cost. Use for predictable cost or shared App Service resources.

In my project: used Consumption Plan for shareholding recalculation — infrequent (triggered by batch uploads), so Consumption was perfect. Cost was near-zero between batch runs.

INTERVIEW TIP: Lead with "scales to zero" and "pay-per-execution" as the core value proposition. Then proactively acknowledge the cold start trade-off — interviewers almost always probe this.

Q10

Azure Functions / Security

How do you handle authentication and authorization in an Azure Function App?

ANSWER

Azure Functions support multiple auth patterns depending on the use case.

1 — Function-Level Auth (built-in key-based):

AuthorizationLevel.Anonymous — open endpoint, no key required.

AuthorizationLevel.Function — requires function-specific API key in x-functions-key header.

AuthorizationLevel.Admin — requires master host key.

2 — JWT Bearer Auth (for user-facing APIs):

Register auth middleware in Program.cs exactly as in a Web API.

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(...)
```

Decorate functions with [Authorize] attribute — works identically to Web API controllers.

3 — Azure AD / Managed Identity (enterprise standard):

Enable Easy Auth in Function App blade → Authentication → Add Microsoft identity provider.

Azure AD validates the token before the function executes — zero auth code in the function.

Service-to-service: calling service gets a token from Azure AD and passes as Bearer. Function validates via Azure AD.

4 — Role-Based Access:

Check role claims inside the function body.

```
User.Claims.Any(c => c.Type == "roles" && c.Value == "Admin")
```

What I use in production:

Public webhook endpoints — AuthorizationLevel.Function (key rotated regularly).

Internal microservice calls — Managed Identity + Azure AD (no secrets shared between services).

User-facing endpoints — JWT Bearer + [Authorize] + role claims.

INTERVIEW TIP: Distinguish the three layers — function key (infrastructure), JWT Bearer (application), Managed Identity (service-to-service). Knowing all three signals senior Azure expertise.

Q11

Angular / Azure

How do you deploy an Angular app as a Web App or Function App in Azure?

ANSWER

Angular apps produce static files after build — they deploy differently from APIs.

Option A — Azure Static Web Apps (recommended for SPAs):

Purpose-built for static frontends + optional serverless API backends.

Free tier available. Built-in CI/CD from GitHub or Azure DevOps.

Global CDN distribution out of the box.

Steps: `ng build --configuration production` → outputs `dist/[app-name]/` → create Azure Static Web App resource → configure `AzureStaticWebApp@0` task with `app_location "/"` and `output_location "dist/[app-name]"`.

Option B — Azure App Service (Web App):

Use when you need SSR (Angular Universal) or custom domain/SSL with more control.

Steps: `ng build` → deploy `dist/` via `AzureWebApp@1` task → add `web.config` URL rewrite so all routes serve `index.html` (required for Angular's client-side router).

Function App — NOT recommended for Angular:

Function Apps are compute (serverless functions), not static file hosts. Deploying an Angular SPA to a Function App is an anti-pattern.

INTERVIEW TIP: Static Web Apps (recommended) or App Service. Clarifying that Function Apps are for backend code shows solid Azure architecture understanding — many candidates miss this distinction.

Q12

Azure DevOps / CI

Steps to deploy an Angular app to a Web App using CI/CD — what goes into the CI pipeline?

ANSWER

CI Pipeline (Continuous Integration) — build, test, and produce the deployable artifact.

```
trigger:
  branches:
    include: [main, develop]
pool:
  vmImage: 'ubuntu-latest'
```

- task: NodeTool@0

inputs:

versionSpec: '20.x'

- script: npm ci

displayName: 'Install packages (clean install — uses package-lock.json exactly)'

```
- script: npx ng lint
displayName: 'ESLint'

- script: npx ng test --watch=false --browsers=ChromeHeadless --code-coverage
displayName: 'Unit Tests (Karma)'

- task: PublishTestResults@2
inputs:
testResultsFormat: 'JUnit'

- script: npx ng build --configuration production
displayName: 'Angular Production Build'

- task: PublishBuildArtifacts@1
inputs:
PathtoPublish: 'dist/[app-name]'
ArtifactName: 'angular-drop'
```

The CD pipeline picks up the 'angular-drop' artifact and deploys it to Azure Static Web Apps or App Service.

INTERVIEW TIP: "npm ci over npm install in CI" — shows you understand reproducible builds. This detail separates senior candidates from mid-level.

Q13

Angular Build

Once you build it, what does the build step give you?

ANSWER

The Angular build produces a build artifact — the `dist/` folder containing optimised static files.

The CI pipeline packages `dist/[app-name]/` and stores it in Azure Pipelines artifact storage as 'angular-drop'. The CD pipeline consumes this artifact and deploys it to the target environment.

The artifact is environment-agnostic — the same compiled output is deployed everywhere. Environment-specific config (API URLs, feature flags) comes from runtime environment variables or a runtime-loaded config file, never baked into the Angular build.

Q14

Angular Build

What does the build folder contain — Angular files or something else?

ANSWER

After `ng build --configuration production` the `dist/[app-name]/` folder contains compiled, optimised static files — not raw Angular TypeScript or component source files.

Contents:

`index.html` — the entry HTML shell. Angular bootstraps from here.

`main.[hash].js` — the compiled application bundle. All components, services, and modules compiled from TypeScript to JavaScript, tree-shaken and minified.

`polyfills.[hash].js` — browser compatibility shims.

`styles.[hash].css` — compiled global CSS (from SCSS/SASS if used).

`[chunk].[hash].js` — lazy-loaded route bundles (one JS file per lazy-loaded route/module).

`assets/` — static files copied from `src/assets/`.

What is NOT there: no .ts files, no node_modules, no Angular decorators (compiled away), no template HTML files (compiled into the JS bundle).

The [hash] suffix is cache-busting — the browser caches files permanently since the hash changes when content changes, improving load performance significantly.

Production build also applies: tree-shaking (unused code removed), minification and uglification, AOT compilation (templates compiled at build time for faster initial render), and differential loading.

Q15

Angular Build

What is the dist folder?

ANSWER

The dist/ folder (short for "distribution") is the output directory of the Angular build process.

It is the production-ready deliverable — everything a browser needs to run your Angular application, and nothing more.

Location: [project-root]/dist/[app-name]/ by default. Configurable in angular.json via outputPath.

The dist/ folder is always listed in .gitignore — it is a build output, never committed to source control.

Deployment: the contents of dist/ are what you serve from any static web server — Azure Static Web Apps, Nginx, App Service, AWS S3 + CloudFront. The server must serve index.html for all routes so Angular's client-side router handles navigation. On App Service this requires a web.config URL rewrite rule.

Q16

C# / Generics

Do you know what a generic class is in C#?

ANSWER

A generic class is a class defined with a type parameter (T) that allows it to operate on any data type while maintaining compile-time type safety.

Why generics exist: code reuse without sacrificing type safety. Avoids boxing/unboxing overhead of non-generic collections (ArrayList stores object; List<T> stores T directly — no heap allocation for value types). Errors are caught at compile-time, not runtime.

```
public class Repository<T> where T : class
{
    private readonly List<T> _store = new();
    public void Add(T item) => _store.Add(item);
    public T Get(int index) => _store[index];
}

// Usage – fully type-safe, no casting:
var repo = new Repository<Customer>();
repo.Add(new Customer { Name = "Ankit" });
Customer c = repo.Get(0); // returns Customer, not object
```

Generic constraints (where clause):

where T : class — T must be a reference type.

where T : struct — T must be a value type.

where T : new() — T must have a parameterless constructor.

where T : IEntity — T must implement IEntity interface.

Real-world usage: BaseRepository<T> with shared SaveChangesAsync(), GetByIdAsync(int id), GetAllAsync(). All domain repositories inherit it with T = their specific entity type.

INTERVIEW TIP: Mentioning constraints (where T : class / IEntity) is what separates a surface-level answer from a senior-level one.

Q17

C# / Generics

Write a generic class with Add and Get methods, and a method that invokes it.

ANSWER

```
// GENERIC CLASS – type-safe in-memory store
public class GenericStore<T>
{
    private readonly List<T> _items = new();

    public void Add(T item)
    {
        if (item == null) throw new ArgumentNullException(nameof(item));
        _items.Add(item);
    }

    public T Get(int index)
    {
        if (index < 0 || index >= _items.Count)
            throw new IndexOutOfRangeException(
                $"Index {index} out of range. Count: {_items.Count}");
        return _items[index];
    }

    public int Count => _items.Count;
}

// METHOD THAT INVOKES THE GENERIC CLASS
public static class GenericStoreDemo
{
    public static void Run()
    {
        // Integer store – no boxing, compile-time safe
        var intStore = new GenericStore<int>();
        intStore.Add(10);
        intStore.Add(20);
        Console.WriteLine(intStore.Get(1)); // Output: 20

        // Custom type store
        var custStore = new GenericStore<Customer>();
        custStore.Add(new Customer { Id = 1, Name = "Ankit" });
        Customer c = custStore.Get(0); // No cast needed
        Console.WriteLine(c.Name); // Output: Ankit
    }
}

public class Customer { public int Id { get; set; } public string Name { get; set; } }
```

Key points to mention: T is resolved at compile time — type-safe, no casting. Null guard in Add() and bounds check in Get() signal production-quality thinking. Works with int, string, Customer, or any type — that is the power of generics.

Wipro

Round 1 · Round 2 · L1 · Apr – Jun 2026

Round 1 — Core Technical Questions

Q01

General & Intro

Tell me about yourself — experience and tech stack?

ANSWER

I'm Ankit, Senior Full Stack Developer, 9 years in .NET Core, Angular, and Azure. Last project: Entity Management System for Grant Thornton (healthcare). Full stack across Angular/React frontend, .NET Core Web API, SQL Server, Azure.

Honest skill ratings if asked: .NET Core — 4/5, Angular — 4/5, Web API — 4/5, EF Core — 3.5/5. Always back ratings with specific project evidence — never rate yourself higher than you can demonstrate.

Q02

Architecture

Abstract class vs Interface — with a real-time scenario?

ANSWER

Abstract class: partial implementation + shared state. Example: BaseRepository with shared DbContext — no need to duplicate SaveChanges/transaction logic in every concrete repository.

Interface: pure contract. Example: IPaymentProcessor, IRepository. Classes implement the contract with their own logic.

Key rule: abstract class = IS-A relationship with shared code. Interface = CAN-DO contract for behaviour. In C# 8+, interfaces can have default method implementations, but abstract classes remain the right choice when shared fields (state) are needed — interfaces cannot have instance fields.

Q03

.NET Core

readonly vs const vs static — when to use each?

ANSWER

const: compile-time constant. Immutable. Implicitly static. Value is baked into IL at compile time. Use for truly fixed literals: MaxRetries = 3, AppName = "MyApp". Cannot be used with complex types.

readonly: set only in constructor or field initialiser. Runtime value. Each instance can hold a different value. Use for DI-injected configuration and settings that vary by environment.

static: belongs to the type, not the instance. Shared across all instances. Combine with readonly for shared-but-immutable config.

Rule of thumb: const for magic literals that never change. readonly for config injected at startup. static for utility/helper methods and shared constants that require runtime initialisation.

Q04

.NET Core

String vs StringBuilder — when does it matter?

ANSWER

string: immutable. Every concatenation creates a new object on the heap. Fine for single concatenations or a handful of operations.

StringBuilder: mutable buffer. Append() is O(1) amortised. Use when concatenating in a loop or building a string from many parts.

```
// BAD — creates N intermediate strings, O(N^2) memory
string result = "";
foreach (var item in items) result += item + ",";

// GOOD — single buffer, O(N) memory
var sb = new StringBuilder();
foreach (var item in items) sb.Append(item).Append(',');
string result = sb.ToString();
```

Rule: fewer than 5 concatenations → string is fine. Any loop or dynamic assembly → StringBuilder. C# 10+ string interpolation is compiler-optimised and often avoids intermediate allocations automatically.

Q05

SQL Server

ROW_NUMBER() vs RANK() vs DENSE_RANK() — differences?

ANSWER

ROW_NUMBER(): unique sequential integer per row regardless of ties. 1, 2, 3, 4, 5. Use for pagination (OFFSET/FETCH pattern).

RANK(): same rank for ties, then skips numbers. Salary 5000, 5000, 4000 → ranks 1, 1, 3 (gap after tie).

DENSE_RANK(): same rank for ties, no gaps. Salary 5000, 5000, 4000 → ranks 1, 1, 2.

For "find Nth highest salary" always use DENSE_RANK() to handle ties correctly:

WITH Ranked AS (

SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk

FROM Employee

)

SELECT DISTINCT Salary FROM Ranked WHERE rnk = 2;

PARTITION BY for per-group ranking (e.g., top earner per department).

Q06

Entity Framework

Eager vs Lazy vs Explicit Loading — and the N+1 problem?

ANSWER

Eager: .Include(x => x.Orders) — loads related data in a single JOIN query. Best for known relationships. This is the primary tool to avoid N+1.

Lazy: auto-loads on property access (requires virtual keyword and EF Core proxies). Risk: N+1. If you loop through 100 entities and access .Orders on each, you fire 100 + 1 queries.

Explicit: manually call .Entry(entity).Collection(x => x.Orders).Load() when needed. Fine-grained control over when related data loads.

```
// BAD — N+1 (100 entities = 101 queries)
var entities = await _db.Entities.ToListAsync();
foreach (var e in entities) { var count = e.Orders.Count; }

// GOOD — Eager: single JOIN query
var entities = await _db.Entities.Include(e => e.Orders).ToListAsync();
```

My rule: disable lazy loading by default. Use eager loading with .Include() for known query shapes. Use explicit loading for conditional load scenarios.

Q07

Angular

Subject vs BehaviorSubject vs ReplaySubject — when to use each?

ANSWER

Subject: no initial value, only future emissions. New subscribers see nothing until the next event. Use for one-shot events: button clicks, route change notifications.

BehaviorSubject: holds the current value. New subscribers receive it immediately. Perfect for shared state that always has a value: authenticated user, dark mode flag, selected entity. The most commonly used Subject type in Angular services.

ReplaySubject(n): replays the last n values to new subscribers. Use when late subscribers need history: notification feed, undo stack.

Rule of thumb: default to BehaviorSubject for shared state. Subject for fire-and-forget events. ReplaySubject only when replay semantics are genuinely required.

Q08

Angular

Change Detection — Default vs OnPush strategy?

ANSWER

Default: Zone.js triggers change detection on every DOM event, HTTP response, setTimeout, and Promise. Checks every component in the tree on every cycle. Simple but can be slow for large component trees.

OnPush: Angular only checks the component when (a) an @Input() reference changes (mutation of an object does NOT trigger it — must be a new reference), (b) an event originates from the component or its children, (c) an Observable emits via the async pipe, or (d) ChangeDetectorRef.markForCheck() is called explicitly.

@Component({ changeDetection: ChangeDetectionStrategy.OnPush })

With OnPush + immutable data patterns, large list components only re-render when their data genuinely changes. This reduced our main dashboard list component's render time by ~60%.

Angular v17+ Signals: fine-grained reactivity without Zone.js. The component re-renders only when a Signal it reads changes — more precise than even OnPush.

Wipro L1 (18 Jun 2026) — Angular & OOP Deep Dive

Q09

Angular vs React

Core differences between Angular and React — from your experience?

ANSWER

Angular = full-fledged MVC framework. React = UI library for the view layer only.

TypeScript: Angular uses it natively by default. React uses JSX with optional TypeScript.

Data Binding: Angular two-way [(ngModel)]. React one-way — props down, events up.

Change Detection: Angular Zone.js cycle. React Virtual DOM diffing.

Routing: Angular has built-in Router. React needs react-router-dom.

State Management: Angular uses Services + RxJS as built-in patterns. React needs useState/useContext or Redux.

From experience: Angular is better for large enterprise apps with strict structure, multiple teams, and long maintenance horizons. React is better for flexible, component-driven UIs with faster iteration and a smaller team.

Angular OnPush + Signals (v17) close the performance gap with React's Virtual DOM.

Q10

Angular / Guards

Explain Auth Guards in Angular — which ones have you used?

ANSWER

CanActivate: runs before a route activates. Most common use — check JWT, redirect to /login if not authenticated.

CanActivateChild: guards all child routes under a parent. Used to protect the entire /admin section with one guard instead of repeating on every child route.

CanDeactivate: prevents leaving a route. Used on complex forms — "You have unsaved changes. Leave anyway?"

CanLoad: prevents a lazy-loaded module from downloading if not authorised. Stops the HTTP bundle request entirely (CanActivate only blocks navigation, the bundle still loads).

Resolve: pre-fetches data before the component loads. Component receives resolved data via ActivatedRoute.snapshot.data.

Modern Angular 15+ functional guards with inject():

```
export const authGuard: CanActivateFn = () => {  
  const auth = inject(AuthService);  
  return auth.isLoggedIn() ? true : inject(Router).navigate(['/login']);  
};
```

Guards I actively use: CanActivate (JWT check), CanActivateChild (/admin routes), CanDeactivate (unsaved form warning).

Q11

C# / OOP

Can we instantiate an abstract class? Difference from a normal class?

ANSWER

NO — attempting to instantiate an abstract class causes a compile-time error. Abstract classes are incomplete by design: they may contain abstract (bodyless) members that subclasses must implement.

Normal class: can be instantiated directly, no abstract members, complete implementation.

Abstract class: CANNOT be instantiated. CAN have abstract (bodyless) methods subclasses MUST override. ALSO has concrete methods and properties — this is the key advantage over interfaces.

```
public abstract class Animal {
    public string Name { get; set; } // concrete property
    public abstract void MakeSound(); // must be overridden
    public void Breathe() => Console.WriteLine("Breathing..."); // shared behaviour
}

public class Dog : Animal {
    public override void MakeSound() => Console.WriteLine("Woof!");
}

// Animal a = new Animal(); -- COMPILE ERROR
Animal dog = new Dog(); // Valid - polymorphism
dog.MakeSound(); // Calls Dog.MakeSound()
```

Real usage: BaseRepository abstract class provides shared SaveChangesAsync() and DbContext. Each concrete repository implements its own domain-specific queries. You cannot new up a BaseRepository — it has no meaning without a specific entity type.

SQL Performance & Production Incident Response

Q01

SQL Server

Walk through how you diagnosed a slow query in production — exact steps, before/after impact?

ANSWER

Dashboard API was timing out at ~8 seconds. I opened SQL Server Profiler and Activity Monitor simultaneously — Profiler to capture exact SQL, Activity Monitor to see CPU and wait statistics.

I found N+1: the dashboard service was firing 1 SELECT per entity to compute shareholding %, resulting in 80–120 DB round-trips per page load. Each query was fast (<50ms) but $100 \times 50\text{ms} = 5$ seconds before network and rendering overhead.

Fix: rewrote as a single stored procedure with a CTE that pre-aggregated shareholding data using SUM() with GROUP BY EntityID, then joined back to the entity list in one pass.

Result: Execution Plan changed from 100× Index Seek + Key Lookup to 1× Hash Match + Clustered Index Scan. Response time: 8s → 420ms (~95% improvement). Verified with SET STATISTICS TIME ON before and after, cross-checked in Application Insights response time graphs. Shared both screenshots in the PR for the code reviewers.

Q02

SQL Server

What Execution Plan patterns indicate a performance problem?

ANSWER

4 red flags I look for in every execution plan:

Key Lookup (Nested Loop): the NC index found the row ID but had to fetch additional columns from the clustered index via a separate lookup. Fix: use an INCLUDE clause on the NC index to create a covering index. Eliminates the secondary lookup entirely.

Table/Index Scan instead of Seek: the WHERE clause is not SARGable — a function is applied to the indexed column (e.g., LOWER(Name) = 'x', YEAR(CreatedAt) = 2025). The index cannot be used. Fix: redesign the predicate so the column is bare on the left side.

Thick arrows between operators: massive intermediate result sets flowing between plan nodes. Usually indicates stale statistics. Fix: UPDATE STATISTICS [TableName] and verify the estimated vs actual row counts align.

Sort operator with high cost: missing an index aligned with the ORDER BY. Add an index with matching column order and sort direction.

When estimated rows diverge from actual rows by more than 10×, statistics are stale or parameter sniffing is the culprit. I always run SET STATISTICS TIME ON alongside the visual plan for concrete before/after timing proof.

Q03

Microservices

Hard deadline in 3 months, two teams disagree: split the service or stay monolithic. How do you decide?

ANSWER

Default position: monolith, unless there is a concrete operational reason to split.

Decision framework in three questions:

Can the teams deploy independently? If the proposed service has no separate deployment cadence and no independent scaling requirements, the split adds coordination overhead without any operational benefit.

What is the coupling surface? If the new service needs synchronous calls back to the monolith for more than two data entities, it is not truly independent — it is a distributed monolith, which is strictly worse: all the operational complexity of microservices with none of the benefits.

What does the risk budget look like? With 3 months and a fixed launch date, each new failure mode (service discovery failures, distributed tracing gaps, network partitions) costs 1–2 sprints to handle properly.

My recommendation: ship as a well-bounded module inside the monolith with clean internal interfaces (IShareholdingService). After launch, use Strangler Fig to extract when real production traffic data justifies the split.

Frame this to both teams as: "We are not cancelling the split — we are sequencing it correctly."

Q04

Azure / Production

A deploy causes cascading failures. DB schema already migrated forward. First 10 minutes?

ANSWER

First action: contain blast radius, not fix root cause.

Minutes 0–2: Azure Portal → App Service → Deployment Slots → Swap back to the stable slot. Old code tolerates additive schema changes (columns added, not dropped) if we follow expand-contract migrations correctly. This immediately stops new failures from reaching users.

If slot swap is not available: scale instance count down to 1 to reduce concurrent failures and free DB connection pool slots while I investigate.

Minutes 2–5: App Insights → Failures blade → find the first exception in the chain. This is the root cause — not the downstream 503s, which are symptoms.

Minutes 5–8: verify schema compatibility. Are all new columns nullable or have DEFAULT values? Does the old code write to any column the new schema renamed or dropped?

Minutes 8–10: if the error rate is dropping after swap, hold and monitor. Post an incident channel update.

Key principle: old code + new schema is always safe under expand-contract. New code + old schema may not be. Always design migrations so the slot swap back is safe.

Infosys

Round 1 — Technical · Jun 2026

C# Fundamentals, OOP & Security

Q01

Collections

Generic vs Non-Generic Collections — key differences?

ANSWER

Always prefer Generic collections in modern C#.

Non-Generic (System.Collections): ArrayList, Hashtable — store objects, require explicit casting on retrieval, cause boxing for value types (int stored as object = heap allocation = GC pressure). Use only when interfacing with legacy code.

Generic (System.Collections.Generic): List<T>, Dictionary<TKey, TValue>, HashSet<T>, Queue<T> — compile-time type safety, no casting, no boxing for value types, full IntelliSense support.

Interface hierarchy: IEnumerable<T> = read-only, forward-only (expose from service layer). ICollection<T> adds Count, Add, Remove. IList<T> adds index access. IQueryable<T> adds expression-tree support for deferred DB execution.

Return the most restrictive interface that satisfies the caller's needs — it keeps your options open for implementation changes.

Q02

C# Fundamentals

Boxing and Unboxing — what is it, what is the cost, how to avoid it?

ANSWER

Boxing: a value type (int, struct) is wrapped in an object allocated on the managed heap. Unboxing: the object is cast back to the value type — involves a runtime type check and a memory copy.

```
// The problem:
ArrayList list = new ArrayList();
list.Add(42); // Boxing: int promoted to object on heap
int x = (int)list[0]; // Unboxing: runtime check + copy
```

Cost: in tight loops with 100k+ iterations, each box creates one heap object. This floods Gen0 with small short-lived objects and triggers frequent GC collections — measurable latency spikes under load.

How I avoid it:

Use generics — List<int> resolves T at compile time, no boxing ever.

Use concrete types in LINQ projections rather than object casts.

Implement IEquatable<T> on structs so .Equals() avoids boxing.

Watch for hidden boxing: passing int to a method that accepts object, or comparing structs through non-generic interfaces.

Production win: replaced an ArrayList with List<decimal> in a report aggregation service — reduced Gen0 GC collections by approximately 60%.

Q03

C# Fundamentals

ref vs out parameters — difference and real-world use?

ANSWER

Both pass by reference so the caller's variable can be modified, but with different contracts.

ref: the variable MUST be initialised before passing. The method may or may not assign a new value. Two-way flow (caller → method → caller).

out: the variable does NOT need initialisation. The method MUST assign a value before returning. One-way flow (method → caller).

```
// ref - update an existing value in-place
void ApplyDiscount(ref decimal price, decimal percent)
=> price -= price * percent;

// out - TryParse pattern: safe parsing without exceptions
if (int.TryParse(queryParam, out int entityId))
return await _service.GetByIdAsync(entityId);
else
return BadRequest("Invalid entity ID.");
```

Senior context: out is the idiomatic pattern for TryGet/TryParse methods. Exception-as-control-flow is expensive (stack trace generation). Dictionary.TryGetValue(key, out var val) is O(1) and exception-free — always prefer it over a Contains() + indexer pair which reads the dictionary twice.

Q04

OOP

Explain the 4 pillars of OOP as applied in your actual project?

ANSWER

Encapsulation: bundle data and behaviour, hide internal implementation.

The Entity class exposes only necessary properties. The internal shareholding computation (CalculatePercentage()) is private. Consumers call a service method — I changed the calculation formula once without modifying any consumer code.

Abstraction: expose what you do, hide how you do it.

IEntityRepository declares GetByIdAsync(), CreateAsync() etc. The service layer depends on the abstraction, not EF Core. In unit tests: inject a mock — no database required. I later swapped EF Core queries to stored procedures in the infrastructure layer with zero service layer changes.

Inheritance: reuse and extend behaviour.

BaseApiController inherits ControllerBase — all controllers share standardised response helpers (OkResult, NotFoundResult wrappers). BaseEntity with CreatedAt, UpdatedAt, IsDeleted — all domain entities inherit these, auto-populated in a SaveChangesAsync() override.

Polymorphism: same interface, different behaviours resolved at runtime.

IShareholdingCalculator has a Calculate(EntityId) method. DirectHoldingCalculator and IndirectHoldingCalculator implement it differently. A factory resolves the correct implementation at runtime based on entity type. Adding a new holding type = one new class — zero changes to existing code (Open/Closed Principle).

How do you secure some API endpoints while keeping others open?

ANSWER

Secure by default: apply `[Authorize]` at the controller class level so every action is protected unless explicitly opened.

```
[Authorize] // Default – all actions secured
[ApiController]
[Route("api/[controller]")]
public class EntityController : ControllerBase
{
    [HttpGet("public-summary")]
    [AllowAnonymous] // Explicitly open this one
    public async Task<IActionResult> PublicSummary() { ... }

    [HttpGet("{id}")] // Secured by controller-level [Authorize]
    public async Task<IActionResult> GetEntity(int id) { ... }

    [HttpDelete("{id}")]
    [Authorize(Roles = "Admin")] // Additional role requirement
    public async Task<IActionResult> DeleteEntity(int id) { ... }
}
```

This pattern means any new action added by the team is automatically protected — a developer must make a deliberate choice to open an endpoint. The alternative (opt-in `[Authorize]` per action) means forgetting an attribute exposes an unprotected endpoint in production.

Policy-based auth `[Authorize(Policy = "CanEditEntities")]` is even more flexible — policies can combine role checks, claim checks, and custom requirements.

Code First vs DB First — which is more efficient and why?

ANSWER

Code First (my preference for greenfield): define C# entities and `DbContext` → EF generates migrations → creates/updates the DB schema.

Advantages: schema changes are versioned in source control (migrations = Git history of every schema change), the team reviews schema changes in PRs, Fluent API controls all constraints and indexes, one command spins up any environment (`dotnet ef database update`), no manual sync between code and database.

DB First: the schema exists → scaffold entities using `Scaffold-DbContext`. Best when inheriting a mature DBA-managed schema with complex views and stored procedures.

My answer: Code First for new projects. It aligns perfectly with CI/CD — migrations run automatically in the deployment pipeline. In the Grant Thornton project: every schema change was a migration file in the PR, applied via the pipeline. Zero manual SQL scripts ever touched source control.

Virtusa

Round 1 — Technical · 11 Jun 2026

Microservices, Docker & Performance

Q01

Microservices

Monolith to microservices migration — how did you approach it?

ANSWER

Used the Strangler Fig pattern — incremental extraction, never a big-bang rewrite.

Step 1: Identify seams — modules with clear bounded contexts and minimal cross-cutting DB joins. Drew a dependency graph; clusters with dense internal arrows = candidates.

Step 2: Extract one service at a time — started with Notification Service. Lowest coupling: no shared tables, async-only communication, even if it went down the core application kept working.

Step 3: Route traffic via Azure API Management — /api/notifications routed to the new service; all other routes stayed on the monolith. Zero client changes required.

Step 4: Migrate data — created a dedicated notification schema, one-time migration script, then cut over.

Step 5: Remove from monolith — deleted the old notification module after a 2-week parallel run with zero errors reported.

Biggest learning: never extract a service that shares a heavily-written DB table with the monolith. Data coupling is much harder to break than code coupling. We deliberately deferred extracting the reporting module because it JOINed across 12 tables from different bounded contexts.

Q02

Microservices

Biggest challenge in microservices and how did you solve it?

ANSWER

Distributed data consistency: a shareholding update needed to update the Shareholding Service DB AND trigger a notification — both must succeed, or neither. Classic two-phase commit problem without a distributed transaction manager.

Problem: a REST call from Shareholding Service to Notification Service could fail after the shareholding DB was already committed. Classic partial failure that leaves the system in an inconsistent state.

Solution — Outbox Pattern + Azure Service Bus:

Step 1: In the same DB transaction as the shareholding update, write an OutboxEvent record to a local OutboxEvents table. If the transaction rolls back, the event is never written.

Step 2: An Azure Function (Timer trigger, every 30 seconds) polls unprocessed OutboxEvents, publishes them to Azure Service Bus, and marks them as published.

Step 3: Notification Service subscribes to the Service Bus topic. Service Bus retries with exponential backoff up to 10 times on failure, then moves to the dead-letter queue for manual inspection.

Result: guaranteed at-least-once delivery with no cross-service distributed transactions. The notification may be slightly delayed (up to 30s), but it is never lost. The shareholding DB is never rolled back due to a notification failure.

Q03

Docker

Write a Dockerfile for a .NET Core microservice?

ANSWER

```
# Stage 1: Build (uses full SDK image ~700MB – compile only)
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY ["ShareholdingService.csproj", "."]
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app/publish

# Stage 2: Runtime (lean ASP.NET runtime image ~200MB – what gets deployed)
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
EXPOSE 8080
COPY --from=build /app/publish .
USER app
ENTRYPOINT ["dotnet", "ShareholdingService.dll"]
```

Key decisions:

Multi-stage build: SDK used only during compile. Final image contains only the runtime — smaller attack surface and faster pulls.

USER app: non-root container. Security hardening requirement on most enterprise Kubernetes clusters.

EXPOSE 8080: .NET 8 changed the default port from 80 to 8080 for non-root containers. A very common gotcha.

.dockerignore: exclude bin/, obj/, *.user, appsettings.Development.json, secrets. Reduces build context and prevents secrets leaking into the image.

Q04

.NET Core

Why does .NET Core run faster than the old .NET Framework?

ANSWER

Six architectural improvements that compound:

Lean modular runtime: .NET Framework shipped everything (WCF, WebForms, System.Drawing). .NET Core is modular via NuGet — only what you reference is deployed. Smaller footprint = faster startup and lower memory usage.

RyuJIT compiler: replaced old JIT64. Produces better machine code, supports SIMD intrinsics, and significantly improves 64-bit throughput.

Span<T> and Memory<T>: slice operations on arrays and strings without heap allocation. In our API, using Span<T> for CSV line parsing eliminated thousands of intermediate string allocations per request, measurably reducing GC pressure.

Improved ThreadPool and async/await: better work-stealing algorithm, lower thread-switching overhead. More concurrent requests handled with fewer OS threads.

Kestrel HTTP server: purpose-built for performance from scratch. Benchmarks consistently place .NET in the top tier for raw HTTP throughput.

AOT and ReadyToRun (.NET 6+): pre-JIT-compiled code reduces cold-start time significantly — critical for Azure Functions on the Consumption plan.

Real result from my project: migrating from .NET Framework 4.6 → .NET 6 on the same hardware, our API average response time dropped ~30% with zero application code changes.

HCL Optum · Decos Global · Successive Noida

Round 1 — Technical Questions · May – Jun 2026

SQL Server — Keys, Indexes & Query Patterns

Q01

SQL Server

Types of keys in a database?

ANSWER

Primary Key: uniquely identifies each row. Cannot be NULL. One per table. Usually clustered (controls physical sort order of the table).

Foreign Key: references the PK of another table. Enforces referential integrity — prevents orphan rows. Can be NULL (meaning "not yet assigned").

Unique Key: ensures all values in a column are distinct. Unlike PK, allows one NULL per column. Multiple unique constraints per table allowed.

Candidate Key: any column or combination that could serve as the PK (unique + not null). The chosen one becomes PK; the rest remain candidate keys.

Composite Key: PK or FK built from two or more columns. PRIMARY KEY (StudentID, CourseID) on an Enrolment table.

Surrogate vs Natural Key: surrogate (IDENTITY/GUID — system-generated) vs natural (SSN, Email — business-meaningful). I prefer surrogate keys — natural keys change in the real world and breaking FK chains in production is painful.

Q02

SQL Server

Can a foreign key be NULL? How does JOIN behave with NULL FK?

ANSWER

YES — a FK column can be NULL by default. Add NOT NULL to prevent it.

NULL FK means the child row exists without an assigned parent — a valid business state. Example: Employee.DepartmentID = NULL means "not yet assigned to a department."

INNER JOIN — NULL FK rows are EXCLUDED. NULL never equals anything, including itself.

```
SELECT e.Name, d.Name FROM Employee e
```

```
INNER JOIN Department d ON e.DepartmentID = d.DepartmentID
```

-- Employees with NULL DepartmentID do NOT appear

LEFT JOIN — NULL FK rows ARE included. Department columns come back as NULL.


```
SELECT e.Name, d.Name FROM Employee e
LEFT JOIN Department d ON e.DepartmentID = d.DepartmentID
-- Unassigned employees appear; d.Name is NULL
```

Use `COALESCE(d.Name, 'Unassigned')` with `LEFT JOIN` for clean reporting. Using `INNER JOIN` silently drops unassigned rows — a very common reporting bug that goes unnoticed until a business user spots the discrepancy.

Q03

SQL Server

Employee table — how would you approach indexing and query optimisation?

ANSWER

Clustered on `EmployeeID` (PK default) — lookups by PK are $O(\log n)$ B-tree traversal, already fast.

For `WHERE Department = 'Finance'` (full scan without index):

```
CREATE NONCLUSTERED INDEX IX_Emp_Dept ON Employee(Department);
```

For reports: `Name + Salary WHERE department`, ordered by `Salary` — covering composite index:

```
CREATE NONCLUSTERED INDEX IX_Emp_Dept_Salary
ON Employee(Department, Salary DESC)
INCLUDE (Name);
```

This satisfies `WHERE`, `ORDER BY`, and `SELECT` columns in one index — eliminates the Key Lookup.

For name search `LIKE 'Ankit%'`: create `IX_Emp_Name` ON `Employee(Name)`.

Warning: `LIKE '%Ankit%'` with a leading wildcard is NOT SARGable — the index cannot be used. Use Full-Text Search for substring matching.

Address column: low selectivity, rarely a query predicate alone — do NOT index. Every index adds write overhead on `INSERT/UPDATE/DELETE`.

Verification: use actual execution plan (Ctrl+M in SSMS). Target: Index Seek, no Key Lookup, no expensive Sort. Monitor unused indexes with `sys.dm_db_index_usage_stats` — unused indexes waste write throughput.

Q04

React

React hooks you have used — `useState`, `useEffect`, `useContext`, `useMemo`, `useCallback`?

ANSWER

`useState`: component-local state. Form inputs, toggles, loading flags. `const [count, setCount] = useState(0)`.

`useEffect`: side effects on mount/update/unmount — API calls, subscriptions, DOM mutations. Return a cleanup function to avoid memory leaks. Dependency array controls re-run frequency.

`useContext`: global state without prop drilling. Auth token, theme, language — provided via `Context.Provider`, consumed anywhere in the tree without intermediate props.

`useCallback`: memoises a function reference across renders. Prevents child components from re-rendering unnecessarily when a callback is passed as a prop. Recomputes only when listed dependencies change.

`useMemo`: memoises a computed value. Use for expensive transformations on large datasets — shareholding percentage computations on 500-row lists.

Rule: do not reach for `useMemo` or `useCallback` prematurely. Profile first with React DevTools Profiler, then optimise. Over-memoisation actually adds overhead and makes code harder to read.

IEnumerable vs IQueryable — when to use each with EF Core?

ANSWER

IQueryable: deferred execution. LINQ operators (Where, OrderBy, Select) are translated to SQL and run on the DB server. Compose the query, materialise last.

```
// Correct – filter runs in SQL, only matching rows cross the wire
var active = await _db.Entities
    .Where(e => !e.IsDeleted && e.Status == "Active")
    .OrderBy(e => e.Name)
    .ToListAsync();
```

IEnumerable: immediate execution. Loads ALL rows into .NET memory, then filters in-process.

```
// Wrong – loads every row from the table into memory first
IEnumerable<Entity> all = _db.Entities.ToList();
var active = all.Where(e => !e.IsDeleted); // in-memory filter on entire table
```

Rule: use IQueryable inside repositories — build the query, apply all filters, then call ToListAsync() once. Return List<T> or IReadOnlyList<T> from repository methods (materialised). Expose IEnumerable<T> from service layer — callers cannot accidentally add deferred LINQ on top of an already-executed collection.

async/await in C# — how does it actually work?

ANSWER

async marks a method as asynchronous — it may contain await expressions and must return Task, Task<T>, or ValueTask<T>.

await suspends the method at that point without blocking the thread. The calling thread is returned to the thread pool immediately to handle other work. When the awaited operation completes, execution resumes — possibly on a different pool thread, but with the same logical flow.

```
// Synchronous – blocks the thread during DB wait
public Customer GetCustomer(int id)
=> _db.Customers.Find(id); // thread blocked for 50ms

// Asynchronous – thread free during DB wait
public async Task<Customer> GetCustomerAsync(int id)
=> await _db.Customers.FindAsync(id); // thread returns to pool, resumes on completion
```

Why it matters under load: a synchronous server with 100 concurrent requests that each spend 50ms waiting for a DB response needs 100 threads tied up waiting. With async/await, those same 100 concurrent requests may use only 2–3 threads because threads are released during IO and reused. This is the core scalability benefit — not latency, but throughput under concurrency.

Quick Reference

Essential Answers at a Glance · 2026

Core Topics — Scan the Night Before

Self Introduction — 90-second version

I'm Ankit, Senior Full Stack Developer with 9 years in .NET Core, Angular, and Azure. Recent: Entity Management System at Grant Thornton — healthcare domain. Impact: cut dashboard API from 8s → 420ms (~95%) via CTE stored procedure rewrite. Stack: .NET 8 Clean Architecture, Angular 17 standalone components, SQL Server, Azure (Functions, Service Bus, DevOps, App Insights, Key Vault).

Docker Steps — .NET Core App

1) dotnet new webapi -n MyApp. 2) Multi-stage Dockerfile: sdk:8.0 AS build → dotnet publish → aspnet:8.0 AS final. 3) .dockerignore (bin, obj, secrets). 4) docker build -t myapp:v1. 5) docker run -d -p 8080:8080 myapp:v1. 6) curl localhost:8080/weatherforecast to verify. 7) Push to Azure Container Registry (ACR) via Azure DevOps pipeline.

Azure Function — Consumption Plan in 3 lines

Pay-per-execution + GB-second. Scales to zero (zero cost when idle). Auto-scales to 200 concurrent instances. Cold start trade-off: 100ms–2s on first invocation after idle. Free: 1M executions/month. Use for event-driven/sporadic workloads. Premium Plan = no cold start + VNET. Dedicated = always-on + fixed cost.

Function App — Auth Layers

AuthorizationLevel.Function (x-functions-key header) for simple webhook auth. JWT Bearer + [Authorize] attribute for user-facing endpoints (same as Web API). Managed Identity + Azure AD for service-to-service (no secrets shared). Easy Auth (zero code) in the Function App blade for enterprise scenarios.

Angular CI Pipeline — 7 steps

1) NodeTool@0 (Node 20.x). 2) npm ci (deterministic, uses package-lock.json exactly). 3) ng lint. 4) ng test --watch=false --browsers=ChromeHeadless. 5) ng build --configuration production → outputs dist/. 6) PublishBuildArtifacts@1 (artifact name: 'angular-drop'). CD pipeline picks up 'angular-drop' and deploys to Azure Static Web Apps or App Service.

dist/ Folder — What's Inside

Compiled static files: index.html (app shell), main.[hash].js (all components + services compiled, tree-shaken, minified, AOT-compiled), polyfills.[hash].js, styles.[hash].css, [chunk].[hash].js (lazy-loaded route bundles), assets/. NO .ts files. NO node_modules. [hash] = cache-busting. NOT committed to source control.

Generic Class — Quick Pattern

```
public class GenericStore { private List _items = new(); public void Add(T item) => _items.Add(item ?? throw new
ArgumentNullException(nameof(item))); public T Get(int i) => (i >= 0 && i < _items.Count) ? _items[i] : throw new
IndexOutOfRangeException(); } // var store = new GenericStore(); store.Add(c); Customer c = store.Get(0); — type-safe,
no cast.
```

N+1 Problem — Detect & Fix

N+1: 1 query fetches parent list → N individual queries fetch each child (lazy loading in a loop). 100 entities = 101 DB round-trips. Detect: SQL Profiler shows hundreds of tiny identical queries. App Insights shows high dependency count. Fix: .Include(e => e.Orders) produces a single JOIN query. Prevention: disable lazy loading by default. Always review EF Core generated SQL in development.

JWT Auth — .NET Core Setup

AddAuthentication(JwtBearerDefaults).AddJwtBearer(opts → TokenValidationParameters with issuer, audience, lifetime, SymmetricSecurityKey). [Authorize] at controller level — secure by default. [AllowAnonymous] on specific open endpoints. Role: [Authorize(Roles = 'Admin')]. Policy: [Authorize(Policy = 'CanEdit')]. Generate: SecurityTokenDescriptor + JwtSecurityTokenHandler.WriteToken().

Strangler Fig — Migration Pattern

1) Identify seam: clear bounded context, minimal cross-cutting DB joins. 2) Extract lowest-coupling service first (Notification — async, no shared tables). 3) Route via Azure API Management. 4) One-time data migration script. 5) 2-week parallel run. 6) Delete old module from monolith. Never extract services sharing heavily-written tables.

Expand-Contract — Safe DB Migrations

Deploy 1 (Expand): add new column with DEFAULT value. New code writes to both old and new column. Old code still works perfectly. Deploy 2 (Contract): remove old column + remove dual-write code. This pattern guarantees that swapping a slot back to old code is always safe — old code + new schema is never broken.

Outbox Pattern — Distributed Consistency

Problem: update DB + notify another service — partial failure if notification call fails. Solution: (1) Write to OutboxEvents table in same DB transaction as the business update. (2) Azure Function (Timer trigger) polls OutboxEvents, publishes to Service Bus, marks as published. (3) Subscriber retries via Service Bus (exponential backoff, dead-letter after N failures). Result: at-least-once delivery, no distributed transactions.

NeoSoft

Round 1 — Technical Interview · 14 July 2026

★ *.NET, Software Architecture & React — first-round screening. Covers architecture fundamentals, EF Core, DI, C# extension methods, React hooks, and SQL ACID properties.*

.NET & Software Architecture

Q01

General & Intro

Total years of experience — overall, and specifically in React?

ANSWER

Lead with total years, then split out the React-specific slice so the interviewer can gauge depth in each: e.g. “9 years overall in full-stack .NET development; ~4 of those years hands-on with React on the frontend alongside Angular.” Naming both frameworks signals versatility rather than a one-trick stack.

(Substitute your own numbers — the structure of the answer matters more than the exact figures.)

INTERVIEW TIP: Never round up — if pressed on a specific React feature, an inflated number becomes a liability.

Q02

General & Intro

Give a brief introduction of yourself.

ANSWER

90-second structure: (1) role + total experience, (2) core stack, (3) most recent project + one measurable outcome, (4) what you're looking for. Example: “Senior Full Stack Developer, 9 years in .NET Core, Angular/React, and Azure. Most recently built an Entity Management System handling shareholding calculations for a healthcare client — cut a dashboard API from 8s to 420ms by replacing N+1 queries with a single CTE stored procedure.”

INTERVIEW TIP: End on the impact metric, not a job-title list — metrics are what get remembered.

Q03

Projects

Tell me about your previous/recent project and the tech stack.

ANSWER

Frame as: domain → your specific ownership → stack → one technical challenge you solved. Example stack: .NET 8 Web API (Clean Architecture: Controller → Service → Repository → EF Core → SQL Server), React/Angular frontend, Azure (App Service, Service Bus, Blob Storage, DevOps CI/CD).

Always name ONE concrete metric (response time, throughput, defect reduction) — it separates a senior answer from a job-description recitation.

Q04

.NET / Clean Architecture

Explain Clean Architecture.

ANSWER

Concentric layers with the Dependency Rule: dependencies only point **inward**, toward the domain. Core (Entities + business rules) has zero framework references. Application layer holds use-cases/interfaces (IRepository, ICommand handlers). Infrastructure implements those interfaces (EF Core, external APIs). Presentation (API/Controllers) is the outermost, replaceable shell.

Payoff: the domain and business logic are testable without a database, a web server, or any framework — swap EF Core for Dapper, or REST for gRPC, without touching business rules.

INTERVIEW TIP: Say the phrase 'Dependency Rule' explicitly — it's the one term that proves real understanding vs. buzzword familiarity.

Q05

.NET / Clean Architecture

What does a typical Clean Architecture folder structure look like?

ANSWER

```
src/  
MyApp.Domain/ -> Entities, ValueObjects, domain events, no external deps  
MyApp.Application/ -> Interfaces, DTOs, CQRS commands/queries, validators  
MyApp.Infrastructure/ -> EF Core DbContext, repository impls, external service clients  
MyApp.API/ -> Controllers, middleware, DI wiring, Program.cs
```

Reference direction: API → Application → Domain, and Infrastructure → Application → Domain. Domain never references Infrastructure or API — enforced with project references, not just convention.

Q06

Architecture Comparison

Difference between Clean Architecture and Microservices?

ANSWER

Different axes, not competitors. Clean Architecture is an **internal code organization** pattern for a single deployable — about dependency direction and separation of concerns within one process.

Microservices is a **system/deployment** pattern — splitting an application into independently deployable, independently scalable services communicating over the network (HTTP/gRPC/message bus), each with its own data store.

In practice: a single microservice is very often built *internally* using Clean Architecture. You can have Clean Architecture in a monolith with no microservices at all.

Q07

Database

What type of database connection did you use in your previous project?

ANSWER

Connection-pooled ADO.NET/SQL Server connections managed transparently by EF Core's DbContext, configured via connection string in appsettings + Azure Key Vault for secrets in production. DbContext registered as **Scoped** (one per HTTP request) so the underlying connection is opened lazily on first query and closed when the request scope disposes.

For read-heavy dashboards, added a secondary read-only connection string pointed at a replica to offload reporting queries from the OLTP primary.

Q08

Entity Framework

How do we establish a connection with Entity Framework?

ANSWER

Register the DbContext in DI: `services.AddDbContext<AppDbContext>(opts =>`

`opts.UseSqlServer(connectionString))` in Program.cs. EF Core does NOT open a physical connection at this point — it's deferred until the first query executes (e.g. `.ToListAsync()`).

The connection string itself is resolved from configuration (appsettings.json → environment variables → Key Vault, in override order). Under the hood, EF Core uses the ADO.NET connection pool, so repeated requests reuse pooled physical connections rather than opening new TCP connections each time.

Q09

Dependency Injection

What is dependency injection?

ANSWER

A pattern where a class receives its dependencies (interfaces) from an external container rather than constructing them itself (`new SomeService()`). Enables loose coupling, unit testing via mocks, and centralized lifetime management.

In .NET, the built-in `IServiceCollection` container resolves the full dependency graph at startup and injects concrete implementations wherever an interface is requested in a constructor.

Q10

DI Lifetimes

Are we using Singleton or Transient? What DI approach/architecture are we using?

ANSWER

Rule of thumb by lifetime: **Singleton** — one instance for the app's lifetime; use for stateless, thread-safe services (e.g. configuration wrappers, HttpClientFactory-backed clients). **Scoped** — one instance per HTTP request; the default for DbContext and anything touching request-specific data. **Transient** — a new instance every time it's resolved; use for lightweight, stateless services with no shared state.

Typical project setup: Repositories/Services registered **Scoped** (tied to DbContext lifetime to avoid “cannot access disposed context” errors), utility/helper classes Transient, and cross-cutting infra like caching clients Singleton. Constructor injection is the standard DI pattern used throughout.

INTERVIEW TIP: Never inject a Scoped/Transient service into a Singleton via constructor — it silently ‘captures’ the first scope. Use `IServiceScopeFactory` inside the singleton instead.

Q11

C# / Extension Methods

Have you heard of extension methods? What are they?

ANSWER

A static method on a static class that appears to “extend” an existing type — including types you don't own (BCL types, sealed classes, third-party libraries) — without subclassing or recompiling the original type.

Syntactic sugar: the compiler rewrites `myString.IsValidEmail()` into `StringExtensions.IsValidEmail(myString)` at compile time. LINQ (`.Where()`, `.Select()`) is itself entirely built on extension methods over `IEnumerable<T>`.

Q12 C# / Extension Methods

Write an example of a static extension method.

ANSWER

```
public static class StringExtensions
{
    public static bool IsValidEmail(this string input)
    {
        if (string.IsNullOrEmpty(input)) return false;
        return System.Text.RegularExpressions.Regex
            .IsMatch(input, @"^[^@\s]+@[^@\s]+\.[^@\s]+$");
    }
}
```

Q13 C# / Extension Methods

Explain how it works.

ANSWER

Three requirements: (1) the class must be **static**, (2) the method must be **static**, (3) the first parameter is prefixed with the **this** keyword — that parameter identifies the type being extended.

The compiler recognizes the *this* modifier and allows the method to be called using instance-method dot-syntax on any value of that type, while under the hood it's still a plain static method call — no reflection, no runtime cost beyond a normal static call.

Q14 C# / Extension Methods

How would you use that extension method in another class?

ANSWER

Just add a **using** directive for the namespace containing the static class — no instantiation needed. Once imported, the extension method is available on every instance of the extended type in that file.

```
using MyApp.Extensions; // namespace containing StringExtensions

if (email.IsValidEmail())
{
    // proceed
}
```

INTERVIEW TIP: If two extension methods with the same signature are in scope from different namespaces, the compiler throws an ambiguity error — keep extension namespaces narrow and purposeful.

React & Frontend Concepts

Q15 React Hooks

What are the different types of hooks in React?

ANSWER

State: `useState`, `useReducer`. Lifecycle/side-effects: `useEffect`, `useLayoutEffect`. Context: `useContext`. Performance/memoization: `useMemo`, `useCallback`. Refs/DOM: `useRef`, `useImperativeHandle`. Concurrent features: `useTransition`, `useDeferredValue`, `useId`, `useSyncExternalStore`. Plus custom hooks composed from the above.

Q16**React Hooks**

Could you explain them all?

ANSWER

useState — local component state, triggers re-render on update. **useEffect** — side effects (API calls, subscriptions) after paint; cleanup function prevents leaks. **useLayoutEffect** — same as useEffect but fires synchronously before paint; use only for DOM measurements that must happen before the browser repaints. **useContext** — reads a Context value, avoids prop drilling. **useMemo** — memoizes a computed value. **useCallback** — memoizes a function reference to prevent unnecessary child re-renders. **useRef** — a mutable box that persists across renders without causing a re-render when changed; also used for direct DOM node access.

Q17**React Hooks**

What is useMemo?

ANSWER

Memoizes the **result** of an expensive computation, recalculating only when one of the listed dependencies changes — otherwise it returns the cached value from the previous render.

```
const sortedList = useMemo(
  () => items.slice().sort((a, b) => a.value - b.value),
  [items]
);
```

Use it when the computation is measurably expensive (large sorts/filters/derived data) — not as a default wrapper on every derived value, since useMemo itself has a small overhead.

INTERVIEW TIP: Profile with React DevTools before reaching for useMemo — premature memoization adds complexity without measurable benefit.

Q18**React / Auth**

I get a JWT token from an API — what's the usual approach to store/set it up in the React app?

ANSWER

Store the token in memory (React state/Context) as the primary approach for SPA session handling — safest against XSS since it's never persisted to storage that scripts can read across page loads. Pair it with an **HttpOnly, Secure, SameSite=Strict** cookie issued by the backend for refresh-token persistence, so a page reload can silently re-authenticate without exposing the token to JS.

Then attach it to outgoing requests via an Axios/Fetch **interceptor** that reads it from a central AuthContext/AuthService rather than each call manually setting headers.

Q19**React / Auth*****Is there any other approach?*****ANSWER**

localStorage/sessionStorage — simplest to implement, persists across reloads, but vulnerable to XSS since any injected script can read it. Acceptable for low-risk internal tools, avoided for anything handling sensitive data.

Redux/Context global store — same in-memory idea as Q18 but centralized for app-wide access, often combined with redux-persist (encrypted) if persistence across reloads is required.

Enterprise pattern: BFF (Backend-for-Frontend) that keeps the token server-side entirely and issues only a session cookie to the browser — the React app never touches the raw JWT at all.

Q20**React / HTTP*****What is an interceptor?*****ANSWER**

A middleware function (Axios: `axios.interceptors.request.use / response.use`) that runs automatically before every outgoing request or after every incoming response, without needing to repeat that logic at each call site.

Q21**React / HTTP*****What do we usually do inside an interceptor?*****ANSWER**

Request interceptor: attach the `Authorization: Bearer <token>` header, inject correlation/trace IDs, set default headers/timeouts.

Response interceptor: centralize error handling — e.g. catch 401 responses, attempt a silent token refresh and retry the original request once, otherwise redirect to login; normalize error shapes; log to monitoring.

Database Management (SQL)**Q22****SQL / Transactions*****What are the ACID properties in SQL?*****ANSWER**

Atomicity — a transaction's operations all succeed or all roll back; no partial writes.

Consistency — a transaction moves the database from one valid state to another, respecting all constraints, triggers, and cascade rules.

Isolation — concurrent transactions don't see each other's uncommitted intermediate state (governed by isolation levels: Read Committed, Repeatable Read, Serializable, Snapshot).

Durability — once committed, changes survive a crash (write-ahead log flushed to disk before acknowledging commit).

Persistent Systems

Round 1 — Technical Interview, 12 July 2021

★ Full-stack breadth round — React internals/performance, Node.js, REST API design, SQL Server, Azure cloud services, and Git workflow, closing with a live coding task.

Background & Experience

Q01

General

How many total years of experience do you have?

ANSWER

State the number directly, then immediately anchor it to the stack: e.g. “9 years, primarily full-stack .NET/React/Azure, with the last few years focused on cloud-native, performance-critical systems.”

Q02

Databases

Do you have experience with databases?

ANSWER

Confirm the relational engine (SQL Server / PostgreSQL) plus ORM (EF Core), and name concrete work: schema design, indexing/query tuning, stored procedures, and any NoSQL exposure (MongoDB/Cosmos DB) for schema-flexible workloads.

Q03

Cloud

Do you have experience on the cloud side?

ANSWER

Name the primary cloud (Azure/AWS) and categorize services used: compute (App Service/Functions/EC2), storage (Blob/S3), messaging (Service Bus/SQS), identity (Azure AD/IAM), and CI/CD (Azure DevOps/GitHub Actions). Structuring the answer by category signals systematic cloud thinking, not just a service list.

Q04

Observability

Experience with monitoring/operations tools (Splunk, Grafana)?

ANSWER

If used: describe what you monitored — e.g. Application Insights/Grafana dashboards for P95 latency, error rate alerts, and log aggregation via Splunk/ELK for incident triage.

If not directly hands-on: be honest, but bridge to adjacent experience (Application Insights, CloudWatch) and the underlying concept — metrics, logs, traces (the three pillars of observability).

React — Internals & Performance

Q05

React Internals

How does the virtual DOM manipulation algorithm work in React?

ANSWER

On state/prop change, React builds a new Virtual DOM tree (a lightweight JS object representation of the UI) and **diffs** it against the previous tree using a heuristic $O(n)$ reconciliation algorithm (not the theoretical $O(n^3)$ full tree-diff).

Key heuristics: elements of different types produce entirely new subtrees (no diffing attempted); elements of the same type diff attributes and recurse into children; the **key** prop identifies list items across renders so React can reorder/reuse DOM nodes instead of destroying and recreating them.

Only the minimal set of real DOM mutations computed from that diff is applied — batched together to minimize layout thrash.

Q06

React Fundamentals

Difference between props and state — which is mutable, which is immutable?

ANSWER

Props — passed from parent to child, **read-only/immutable** from the child's perspective; the child must never mutate them directly. Changing UI driven by props means the parent re-renders with new prop values.

State — owned and managed internally by the component via `useState/useReducer`, and is **mutable** — but only through its setter (`setState/dispatch`), never via direct mutation, since React relies on reference-change detection to know a re-render is needed.

Q07

React Internals

How does React Fiber improve rendering performance?

ANSWER

Fiber (React 16+) reimplemented the reconciler as an **incremental, interruptible** unit-of-work linked-list data structure instead of the old synchronous recursive stack-based reconciler.

This lets React pause rendering mid-tree to yield to the browser for higher-priority work (user input, animations), then resume later — preventing long renders from blocking the main thread and causing jank.

It also enables **priority-based scheduling**: urgent updates (typing, clicks) preempt low-priority ones (data fetching results, offscreen content) — the foundation for concurrent features like `useTransition`.

Q08

React Hooks

Difference between `useEffect`, `useLayoutEffect`, and `useMemo`?

ANSWER

useEffect — runs asynchronously *after* the browser paints. Use for side effects that don't need to block visual output: API calls, subscriptions, analytics.

useLayoutEffect — runs synchronously *before* the browser paints, blocking paint until it completes. Use only when you must measure/mutate the DOM before the user sees it (e.g. reading `getBoundingClientRect` to reposition a tooltip) — overusing it causes visible jank.

useMemo — not an effect at all; it memoizes a *computed value* during render to avoid recomputation, with no side effects or DOM interaction involved.

Q09

React Performance

React is slow due to excessive re-rendering — how do you troubleshoot and fix it?

ANSWER

- 1) **Profile first** — React DevTools Profiler to record a session and see which components render, how often, and why (“why did this render” flags).
- 2) **Identify the cause** — common culprits: new object/array/function literals created inline as props on every render (breaks reference equality), missing/incorrect dependency arrays, Context value changing reference every render forcing all consumers to re-render, unkeyed or badly-keyed lists.
- 3) **Fix targeted, not global** — wrap components in `React.memo` where prop equality is meaningful, stabilize function props with `useCallback` and derived values with `useMemo`, split large Context into narrower ones, virtualize long lists (react-window).
- 4) **Re-measure** — confirm the fix with another Profiler pass rather than assuming improvement.

Q10

React Performance

A child re-renders even though its props haven't changed — cause and fix?

ANSWER

Root cause is almost always **referential inequality**, not “React being wrong”: the parent passes a new object/array/function literal on every render (e.g. `onClick={() => doThing()}` or `style={{color:'red'}}`), so shallow prop comparison sees a “new” value even though the content is identical.

Fix: wrap the child in **React.memo**, and stabilize the parent's props with **useCallback** (for functions) and **useMemo** (for objects/arrays) so the same reference is passed across renders when the underlying data hasn't changed.

Also check Context: if the child consumes a Context whose Provider value is re-created each render, every consumer re-renders regardless of memoization — fix by memoizing the Provider's value.

Node.js & REST APIs

Q11

Node.js

How does the event loop work in Node.js?

ANSWER

Node runs JS on a **single thread** but delegates I/O to the OS/libuv thread pool, using the event loop to coordinate async callbacks without blocking.

Phases per tick: **timers** (setTimeout/setInterval callbacks) → **pending callbacks** → **poll** (retrieves new I/O events, executes their callbacks) → **check** (setImmediate) → **close callbacks**. Microtasks (Promise .then, process.nextTick) drain *between every phase* — process.nextTick has even higher priority than Promise microtasks.

Practical implication: a CPU-bound synchronous loop blocks the entire event loop — no other request is served until it completes, which is why CPU-heavy work is offloaded to worker_threads or a separate service.

Q12

Node.js

Difference between require and import?

ANSWER

require — CommonJS, synchronous, dynamic (can be called conditionally/anywhere in code), resolved at runtime, returns `module.exports`.

import — ES Modules, statically analyzed at compile time (hoisted, must be top-level in standard usage), supports async `dynamic import()` and tree-shaking by bundlers since the dependency graph is known ahead of execution.

Node supports both today (via `.mjs/.cjs` extensions or "type" in `package.json`) but they have different resolution algorithms and can't be freely mixed without interop rules.

Q13

REST APIs

Difference between PUT and PATCH?

ANSWER

PUT — replaces the *entire* resource with the payload provided; idempotent (same request repeated produces the same end state); omitted fields are typically cleared/reset.

PATCH — applies a *partial* update, modifying only the fields included in the payload; not guaranteed idempotent by spec (though most implementations make it so) since repeated partial patches with relative changes could differ.

Q14

REST / Security

How do you secure RESTful web services?

ANSWER

AuthN/AuthZ — JWT Bearer tokens validated on every request; [Authorize] secure-by-default at the controller level, role/policy-based checks for fine-grained access.

Transport — HTTPS/TLS everywhere, HSTS enabled.

Input validation — model validation/DTOs to prevent injection and malformed payloads; parameterized queries against SQL injection.

Rate limiting & throttling — API Management/gateway level to prevent abuse and brute-force attempts.

CORS — explicit allow-listed origins, never wildcard with credentials.

Secrets — never hardcoded; pulled from Key Vault/Secrets Manager via managed identity.

Auditing — structured logging of auth failures and sensitive operations for traceability.

SQL Server

Q15

SQL Server

Difference between stored procedures and triggers?

ANSWER

Stored procedure — explicitly invoked (EXEC ProcName) by application code or another script; encapsulates reusable business logic/queries, can accept parameters and return result sets.

Trigger — implicitly fires automatically in response to a DML event (INSERT/UPDATE/DELETE) on a specific table; cannot be called directly and has no parameters. Used for enforcing business rules, auditing changes, or maintaining denormalized data — but overuse makes data flow hard to trace, so most teams prefer explicit application-layer logic or stored procedures where possible.

Azure Cloud

Q16

Azure

Which Azure services have you used?

ANSWER

Organize by category rather than listing flatly: **Compute** — App Service, Azure Functions, Container Apps/AKS. **Storage** — Blob Storage, Azure SQL Database. **Messaging** — Service Bus (topics/subscriptions for pub-sub). **DevOps** — Azure DevOps YAML pipelines, Container Registry. **Monitoring** — Application Insights. **Security** — Key Vault (secrets via managed identity), Azure Active Directory.

Q17

Azure AD

Have you used Active Directory in Azure?

ANSWER

Yes — for enterprise SSO via Easy Auth on App Service/Function Apps (zero auth code in the app itself, Azure AD validates the token before the request reaches the function), and for service-to-service auth using **Managed Identity** so no secrets/credentials are shared between internal services.

Also used for role-based access control — checking group/role claims from the AD token to gate specific endpoints or actions.

Version Control

Q18

Git

Do you have experience with Git, Bitbucket, or GitHub?

ANSWER

Confirm the platform(s) and describe the workflow used: feature-branch/trunk-based development, PR review gates, branch protection rules requiring passing CI + approvals, and squash-merge vs. merge-commit conventions.

Q19**Git**

What is a merge conflict and how do you resolve it?

ANSWER

Occurs when Git can't automatically reconcile changes because the **same lines** in a file were modified differently on two branches being merged — Git marks the file with conflict markers (<<<<<<, =====, >>>>>>) and pauses the merge.

Resolution: open each conflicted file, manually choose/combine the correct content, remove the conflict markers, then *git add* the resolved file and *git commit* to finalize the merge (or *git rebase --continue* if resolving during a rebase).

Prevention: smaller, frequent commits/PRs, pulling/rebasing from main regularly, and clear ownership boundaries between files reduce the frequency and size of conflicts.

Practical Task**Q20****Practical Coding**

Write a JavaScript/React snippet to fetch and display data (ID, Name, Email).

ANSWER

```
function UserList() {
  const [users, setUsers] = React.useState([]);
  const [loading, setLoading] = React.useState(true);
  const [error, setError] = React.useState(null);

  React.useEffect(() => {
    let isMounted = true;
    fetch('/api/users')
      .then(res => {
        if (!res.ok) throw new Error('Request failed: ' + res.status);
        return res.json();
      })
      .then(data => { if (isMounted) setUsers(data); })
      .catch(err => { if (isMounted) setError(err.message); })
      .finally(() => { if (isMounted) setLoading(false); });
    return () => { isMounted = false; }; // cleanup avoids state update after unmount
  }, []);

  if (loading) return Loading...;
  if (error) return Error: {error};

  return (
    IDNameEmail
    {users.map(u => (
      {u.id}{u.name}{u.email}
    ))}
  );
}
```

INTERVIEW TIP: The `isMounted` cleanup flag and `key={u.id}` on the mapped row are the two details that signal production habits, not just a tutorial-level fetch.